Server2 notice publishing

S18c implements a notice library, read-only HTTP router and operator CLI. **S20 must mount the router in the service process.** This task does not implement client/web popups, SQLite, deployment, paid translation verification or device delivery.

Prerequisites

Run the compiled `server2-notice` binary from `server2/` so it reads that directory's private `.env`. Reuse `SERVER2_S3_BUCKET`, `SERVER2_S3_REGION` and the existing AWS static credentials or EC2 instance role. Notice state uses `index/v2/notices/state.json`; checked-in S03/S10f policies already allow its conditional reads/writes. Actual role access is a deployment check, not established by the local peers.

Publish-time translation uses `OPENAI_SECRET_KEY`, `TRANSLATION_LLM_MODEL` (default `gpt-6-luna`) and `TRANSLATION_LLM_REASONING_EFFORT` (default `none`; explicit empty omits reasoning). These names come from #2623. No key means no translation request; valid original texts still publish. No model/account access or live translation quality is claimed by local tests. Never print keys or put them into notice JSON.

Publish, inspect and withdraw (N-S1)

An authorized operator supplies plain JSON on stdin. Korean or English is required; supported keys are `en, es, ja, de, pt, fr, ko`. A title is at most 100 Unicode scalar characters and body at most 2000. HTML tags and disallowed controls are rejected. `linkUrl` is optional and must be absolute HTTPS without credentials. Times use RFC3339, normalize to UTC, and endsAt must be later than startsAt; omitted startsAt means publication time.

```
printf '%s' '{"texts":{"en":{"title":"Service update","body":"Weather updates may be
delayed."}},"linkUrl":"https://example.com/status"}' | ./server2-notice add
./server2-notice list
./server2-notice show 1
./server2-notice latest fr-FR
./server2-notice withdraw 1
```

Successful mutations print numeric `id` and an operation identity only after the complete state is durable. Translation warnings name the failed language without exposing source text, key or request URL. A refused/invalid mutation exits nonzero; it never acknowledges an in-memory-only change.

Update or fill missing translations (N-S2, N-S4)

```
printf '%s' '{"linkUrl":"https://example.com/new-status"}' | ./server2-notice update 1
printf '%s' '{"texts":{"en":{"title":"Resolved","body":"Updates are current again."}}}' |
./server2-notice update 1
./server2-notice translate 1
```

A metadata-only update keeps existing texts. Supplying `texts` replaces all language texts with that input and translates missing languages for the new source; include any manual language text you want to keep. Operators cannot supply generated-translation provenance. Korean is the translation source when present, otherwise English. Stored generated texts retain `{from, model}`. `translate` fills only missing languages and never overwrites a concurrent manual text. If the source changes while translation runs, the stale application fails explicitly.

Invalid JSON, unknown fields/languages, overlimit text, markup, invalid time windows or HTTP links leave durable bytes unchanged. Translation refusal, malformed/overlimit output, HTTP failure or a 30-second deadline leaves that language missing and does not prevent the validated original from publishing. Each language gets one attempt; later `translate` is an explicit operator action.

Read API and failures (N-S3)

Once S20 mounts the exported router, `GET /v000903/notice/latest?lang=fr-FR` returns a notice or `{"notice":null}`. Active means startsAt<=now and now<endsAt when supplied, with no withdrawal. Latest startsAt wins; ties use highest numeric ID. Query language overrides Accept-Language. Without a query, supported primary subtags are tried in wire order (q=0 excluded). Fallback is English, then Korean. Response fields are id, title, body, linkUrl, lang, startsAt and endsAt. Headers are `Cache-Control: public, max-age=300` and `Vary: Accept-Language`. There is no HTTP write endpoint.

Every request checks canonical S3. Missing object means no notices. An unreadable/corrupt cold state returns 503 with no-store. After a verified read, a transient/corrupt replacement preserves that copy with a sanitized diagnostic; current time still removes expired notices. This verified memory copy is disposable and cannot prove a write succeeded.

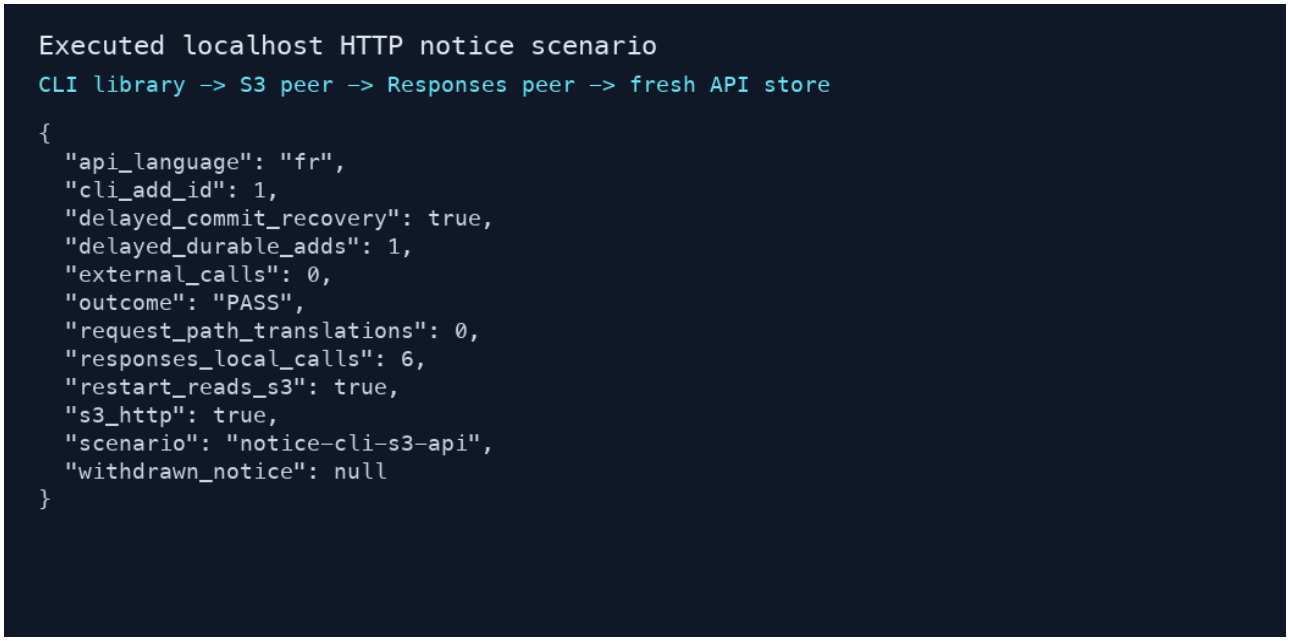
Concurrency, backup and uncertain outcomes (N-S4)

Conditional writes preserve monotonic IDs and operation identities across concurrent CLIs. An uncertain write is read back; the operation identity proves a committed result even after another writer. An unchanged readback leaves the original conditional write pending. Subsequent read failure, retry refusal or retry exhaustion remains uncertain until the operation is found or a changed canonical precondition rules out the old write. Identical retries retain the original body and identity. An unreadable outcome exits nonzero and reports the nonsensitive operation ID. Inspect durable state before repeating the action with a new operation. A restarted process never treats an uncertain local result as success. Storage conflicts have bounded reconciliation; translations have no automatic retries.

S3 Versioning preserves operator history. Do not delete state or blindly restore an older complete version: that can reuse IDs and lose operation identities. Inspect a known older version through an authorized S3 read, then use `update` to restore selected content or `withdraw` to stop publication while keeping current IDs/history. State refuses growth above 8MiB or 10000 records; there is no automatic pruning. Escalate a capacity failure before changing history. The notice key and all mutable state are excluded from weather raw disk caching.

Executed isolated scenario

The actual `notice_smoke` example invokes the CLI command library against an owned HTTP S3 peer, publishes six translations through an owned Responses HTTP peer, starts a fresh read API store, reads French, then withdraws and reads null. A separate HTTP fault peer delays a failed-response PUT, returns an unchanged readback and fails the next read; the CLI/store reports uncertainty and recovers the original identity after the single delayed commit. The binary's production construction uses fixed official OpenAI HTTPS and normal S3 settings; the smoke injects dependencies in code and loads no `.env`. This proves local HTTP contracts, not IAM permissions, paid provider output or a deployed route.




```
cargo run --manifest-path server2/Cargo.toml --example notice_smoke
```

The maintained verification record distinguishes the actual focused/fault checks from later review/CI outcomes. The Responses request format follows the official structured-output guide and reasoning guide. No real paid API request is part of verification.